

KUBERNETES NETWORKING • CONCEPTS & HANDS-ON LABS

Understanding Kubernetes Pods and Pod-to-Pod Networking

What a Pod is, how containers inside one Pod communicate over `localhost`, and why that same approach fails once containers are split across separate Pods.

DURATION

45–60 minutes

DIFFICULTY

Beginner

LABS INCLUDED

2 hands-on labs

TOOLING

kubectl + any cluster

What You Will Learn

- 01** What a Pod is, and why Kubernetes groups containers this way
- 02** How Pod networking works under the hood (the IP-per-Pod model)
- 03** Lab A — verifying that containers in the same Pod share localhost
- 04** Lab B — verifying that containers in different Pods do not share localhost
- 05** How Pod-to-Pod communication actually happens, and where Services fit in

What Is a Pod?

A Pod is the smallest deployable unit in Kubernetes. It is not a container itself, but a thin wrapper around one or more containers that guarantees those containers are scheduled together, on the same node, and share certain resources for as long as the Pod exists.

It helps to think of a Pod as a single logical host. Just as a traditional virtual machine can run several cooperating processes that share one IP address and one filesystem, a Pod allows several containers to share one IP address and, optionally, storage volumes. The difference is that each process inside that logical host is packaged as its own container image, with its own filesystem layer, dependencies, and lifecycle.

What a Pod Provides to Its Containers

- A shared network namespace — one IP address, one loopback interface, and one port space, shared by every container in the Pod.
- Optional shared storage — volumes defined at the Pod level can be mounted into more than one container.
- Co-scheduling — all containers in a Pod are placed on the same node and started as a unit.
- A shared lifecycle — the Pod is created, restarted, and terminated as a whole, even though individual containers within it can restart independently on failure.

Why Kubernetes Uses Pods Instead of Bare Containers

Kubernetes could, in principle, schedule individual containers directly. It does not, because several common patterns require two or more containers to be tightly coupled — placed together, networked together, and started and stopped together. The Pod abstraction exists to make that coupling explicit and reliable. Typical multi-container patterns include:

- Sidecar — a helper container that extends the main container, such as a log shipper that reads files written by the primary application.
- Ambassador — a proxy container that simplifies how the main container reaches external services.
- Init container — a container that runs to completion before the main application container starts, commonly used to perform setup work.

In every one of these patterns, the helper container needs to be reachable from the main container quickly and simply. Shared Pod networking is what makes that possible without any extra configuration.

Single-Container Pods Are the Common Case

In Practice

Most Pods in a real cluster contain exactly one container. Multi-container Pods are a deliberate design choice used for the coupling patterns above — not the default way of running an application. This guide uses a two-container Pod specifically to make the shared-networking behavior observable.

The Pod Networking Model

Kubernetes networking is built on a small set of rules, often called the Kubernetes networking model. Two of those rules matter most for this guide:

- 1 Every Pod is assigned its own unique IP address, and every container within that Pod shares it.
- 2 Pods can communicate with all other Pods in the cluster using those IP addresses directly, without requiring any address translation (NAT) between them.

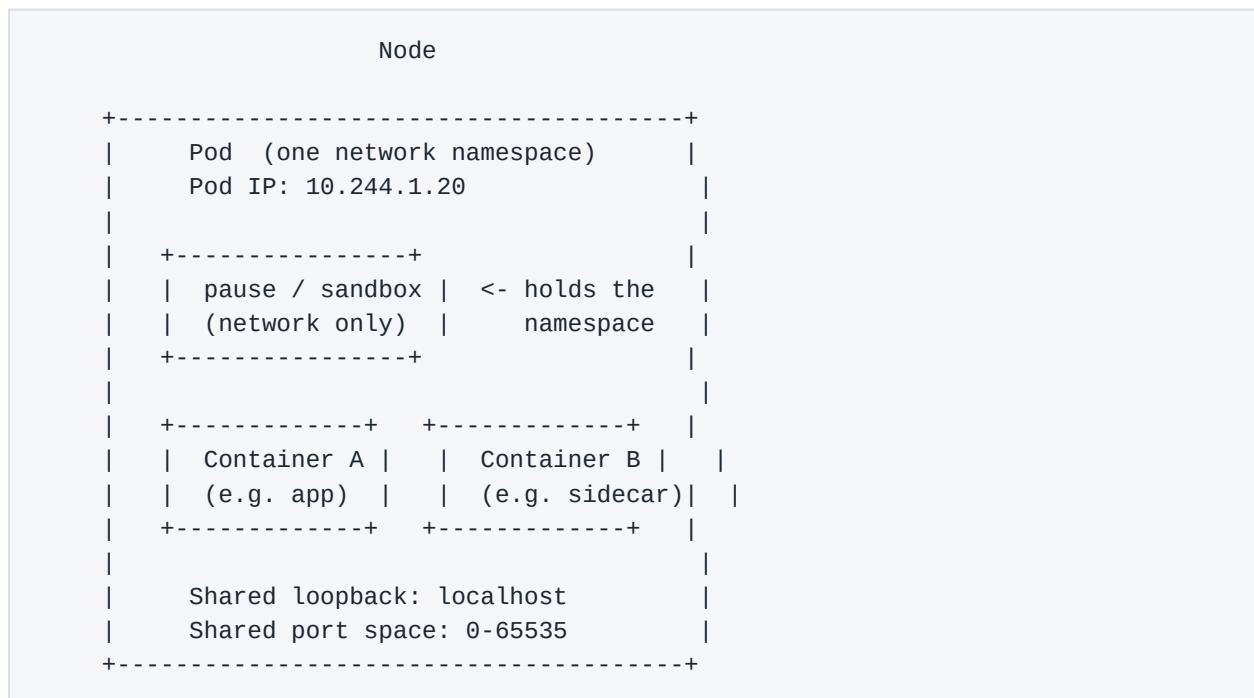
The first rule is what this guide focuses on. It is implemented using a Linux kernel feature called a network namespace — an isolated instance of the networking stack, with its own network interfaces, routing table, and port space. When Kubernetes creates a Pod, it creates one network namespace for that Pod and attaches every container in the Pod spec to it.

Mechanically, this is done using a lightweight container (sometimes called the pause container) whose only job is to hold the namespace open for the lifetime of the Pod. Every application container defined in the Pod is then joined to that same namespace instead of receiving one of its own.

Consequence: Containers in a Pod Behave Like Processes on One Host

Because the network namespace is shared, containers in the same Pod interact exactly the way separate processes on a single Linux host would: they see the same loopback interface, and a port opened by one container is visible to the others as `localhost:<port>`. No cluster networking, DNS, or routing is involved for this kind of communication — it never leaves the kernel's loopback device.

Diagram: Anatomy of a Pod



One Pod IP, one loopback interface, one port space — shared by every container the Pod contains.

What Happens Across Pods

Every Pod receives its own network namespace, and therefore its own IP address. Two containers placed in two different Pods do not share a loopback interface, so `localhost` inside one Pod has no relationship to `localhost` inside another. Reaching a container in a different Pod requires addressing that Pod directly, over the cluster network, using its Pod IP. Lab A and Lab B in this guide are built to make that distinction directly observable.

Lab A — Communication Within a Single Pod

This lab deploys one Pod containing two containers and confirms, from the command line, that both containers share a single IP address and can reach each other over `localhost`.

Objective

- Deploy a Pod that contains two containers.
- Confirm that the Pod reports a single IP address regardless of container count.
- Confirm that one container can reach a service running in the other container using `localhost`.

Prerequisites

- A running Kubernetes cluster and a configured `kubectl` context.
- Outbound access from the cluster nodes to pull the `nginx` and `busybox` images.

STEP 1 Define the Pod

Create **`shared-network.yaml`**. The manifest defines one Pod with two containers: an `nginx` web server, and a `busybox` container that will act as the test client.

```
apiVersion: v1
kind: Pod
metadata:
  name: shared-network
spec:
  containers:
    - name: nginx
      image: nginx

    - name: client
      image: busybox
      command:
        - sleep
        - "3600"
```

STEP 2 Deploy It

```
kubectl apply -f shared-network.yaml
```

STEP 3 Confirm Both Containers Are Running

```
kubectl get pods
```

Expected output

NAME	READY	STATUS
shared-network	2/2	Running

The READY column reads 2/2: one Pod, two containers.

STEP 4 Inspect the Pod IP

```
kubectl get pod shared-network -o wide
```

Example output

NAME	IP
shared-network	10.244.1.20

A single IP address is reported for the Pod as a whole — the structural fact this lab sets out to confirm.

STEP 5 Open a Shell in the Client Container

```
kubectl exec -it shared-network -c client -- sh
```

The `-c client` flag selects the BusyBox container specifically, since the Pod contains more than one container.

STEP 6 Reach nginx over localhost

```
wget -qO- http://localhost
```

Expected output

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
```

Why This Works

The BusyBox image does not include NGINX. The request succeeds because `localhost` resolves within the Pod's shared network namespace, reaching the sibling `nginx` container listening on port 80 of that same namespace.

STEP 7 Confirm with 127.0.0.1

```
wget -qO- http://127.0.0.1
```

Same result as the previous step — `localhost` and `127.0.0.1` both resolve to the loopback interface of the shared namespace.

STEP 8 Confirm Shared Port Visibility

List listening sockets from inside the client container to confirm that port 80, owned by the neighboring `nginx` container, is visible from this container's network view:

```
netstat -tln 2>/dev/null || ss -tln
```

Port 80 appears in the listening socket table even though it was opened by the sibling container — further confirmation that the port space is shared at the Pod level, not isolated per container.

KEY TAKEAWAY

Containers that run inside the same Pod share a single Linux network namespace and, as a result, a single Pod IP address. They communicate with one another over `localhost`, exactly as separate processes on one host would.

Lab B — Communication Between Different Pods

This lab deploys the same two workloads used in Lab A — an NGINX server and a BusyBox client — but places each in its own Pod. The goal is to observe, directly, that `localhost` no longer provides connectivity once the containers are no longer co-located, and that a Pod IP address is required instead.

Objective

- Deploy two independent Pods, each with its own IP address.
- Confirm that `localhost` cannot be used to reach a container in a different Pod.
- Confirm that the target Pod remains reachable using its Pod IP address.

STEP 1 Define the nginx Pod

Create `nginx.yaml`:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx
```

```
kubectl apply -f nginx.yaml
```

STEP 2 Define the busybox Pod

Create `busybox.yaml`:

```
apiVersion: v1
kind: Pod
metadata:
  name: busybox
spec:
  containers:
  - name: busybox
    image: busybox
    command:
    - sleep
    - "3600"
```

```
kubectl apply -f busybox.yaml
```

STEP 3 Confirm Two Pods, Two IP Addresses

```
kubectl get pods -o wide
```

Example output

NAME	READY	STATUS	IP
nginx	1/1	Running	10.244.1.15
busybox	1/1	Running	10.244.2.10

Unlike Lab A, each Pod is assigned its own IP address — the two workloads no longer share a network namespace.

STEP 4 Open a Shell in the busybox Pod

```
kubectl exec -it busybox -- sh
```

STEP 5 Attempt localhost

```
wget -qO- http://localhost
```

Expected output

```
wget: can't connect to remote host (127.0.0.1): Connection refused
```

Why This Fails

BusyBox is only able to see its own Pod's loopback interface. Because `nginx` is not running in this Pod, there is nothing listening on `localhost` here — the request fails as it would between two unrelated hosts.

STEP 6 Reach nginx Using Its Pod IP

Substitute the actual IP address observed in Step 3 for the `nginx` Pod:

```
wget -qO- http://10.244.1.15
```

Expected output

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
```

The BusyBox Pod successfully reaches the NGINX Pod once addressed by its Pod IP. This is the standard communication path between Pods: the Kubernetes networking model guarantees that any Pod can reach any other Pod's IP address directly, without NAT, across the cluster network.

KEY TAKEAWAY

Containers placed in different Pods do not share a network namespace, so each Pod receives its own IP address and its own `localhost`. Reaching a container in another Pod requires addressing that Pod directly by its IP — `localhost` never crosses a Pod boundary.

A Note on Pod IP Stability

Pod IP addresses are not stable. If a Pod is deleted and recreated — including during a routine rolling update — it is very likely to receive a new IP address. Hardcoding a Pod IP, as this lab does for illustration, is not a pattern to use in real workloads. A Kubernetes Service exists to provide a stable address in front of a changing set of Pod IPs; that mechanism is outside the scope of this guide but is the natural next step after these two labs.

PART 4 — REFERENCE

Comparison: Same Pod vs. Different Pods

The table below summarizes the structural difference demonstrated across the two labs.

Property	Lab A — Same Pod	Lab B — Different Pods
Network namespace	One namespace, shared by both containers	One namespace per Pod — not shared
IP addresses assigned	One IP for the whole Pod	One IP per Pod (two total)
localhost reachability	Works — both containers see the same loopback interface	Fails — each Pod has its own loopback interface
Correct way to connect	localhost:	:, or a Service in production
Underlying mechanism	Shared Linux network namespace	Cluster networking (CNI) routes traffic between namespaces

PART 5 — REFERENCE

Troubleshooting

Symptom	Likely Cause	Resolution
Pod stuck in Pending	Insufficient cluster resources, or image still pulling	Run <code>kubectl describe pod</code> and review the Events section.
wget can't connect inside the same Pod (Lab A)	NGINX not yet ready, or command run in the wrong container	Confirm READY shows 2/2 and that -c client was passed to <code>kubectl exec</code> .
wget succeeds over localhost in Lab B (unexpected)	Command was run against the wrong Pod, or an unrelated service is bound to port 80 on that node	Re-verify with <code>kubectl exec -it busybox -- sh</code> and confirm the container name in the prompt.
error: unable to upgrade connection on <code>kubectl exec</code>	Node-level kubelet connectivity issue	Confirm the node is Ready via <code>kubectl get nodes</code> and retry.

PART 6

Cleanup

Remove the resources created across both labs:

```
kubectl delete pod shared-network
kubectl delete pod nginx
kubectl delete pod busybox
```

PART 7

Summary of Key Concepts

- A Pod is the smallest deployable unit in Kubernetes — a thin wrapper that lets one or more containers share a network namespace, and optionally storage, as a single scheduling unit.
- Every Pod is assigned exactly one IP address, regardless of how many containers it contains.
- Containers in the same Pod share a loopback interface and communicate over `localhost`.
- Containers in different Pods each have their own network namespace; reaching across Pods requires the target Pod's IP address.
- Pod IP addresses are not stable across restarts, which is why production workloads typically sit behind a Kubernetes Service rather than being addressed by Pod IP directly.

PART 8

Self-Assessment

Use the following questions to confirm understanding of the material.

- 1 What Linux kernel feature allows containers within a Pod to share a single network identity?
- 2 How many IP addresses are assigned to a Pod that contains three containers?
- 3 In Lab B, why did `wget -q0- http://localhost` fail from inside the BusyBox Pod?
- 4 If a third container were added to the `shared-network` Pod from Lab A and it opened a listener on port 8080, would the existing `nginx` container be able to reach it over `localhost:8080`? Why?
- 5 Why is addressing a Pod directly by its IP address, as done in Lab B, not recommended for production workloads?

Reference answers

1. The Linux network namespace.
2. One — the Pod IP is shared regardless of container count.
3. The BusyBox Pod has its own network namespace and its own loopback interface; nothing in that Pod is listening on localhost, since NGINX runs in a separate Pod.
4. Yes — all containers in a Pod share the same loopback interface and port space, so any port opened by one container is reachable by the others over localhost.
5. Pod IP addresses change whenever a Pod is recreated (including during routine rolling updates), so a hardcoded Pod IP will eventually point to nothing; a Service provides a stable address instead.

What's next: with Pod-level networking established, the natural follow-on topic is the Kubernetes Service object — the mechanism that gives a changing set of Pods a stable address, and the standard way production workloads discover one another.